

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE			DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab		Academic Year:2025-2026
Course Coordinator Name		Venkataramana Veeramsetty		
Instructor(s) Name		Dr. V. Venkataramana (Co-ordinator)		
		Dr. T. Sampath Kumar		
		Dr. Pramoda Patro		
		Dr. Brij Kishor Tiwari		
		Dr.J.Ravichander		
		Dr. Mohammand Ali Shaik		
		Dr. Anirodh Kumar		
		Mr. S.Naresh Kumar		
		Dr. RAJESH VELPULA		
		Mr. Kundhan Kumar		
		Ms. Ch.Rajitha		
		Mr. M Prakash		
		Mr. B.Raju		
		Intern 1 (Dharma teja)		
		Intern 2 (Sai Prasad)		
		Intern 3 (Sowmya)		
		NS_2 (Mounika)		
Course Code	24CS002PC215	Course Title	AI Assisted Coding	
Year/Sem	II/I	Regulation	R24	
Date and Day of Assignment	Week5 - Monday	Time(s)		
Duration	2 Hours	Applicable to Batches		
AssignmentNumber:10.1(Present assignment number)/24(Total number of assignments)				
Q.No.	Question			Expected Time to complete
1	Lab 10 – Code Review and Quality: Using AI to Improve Code Quality and Readability Lab Objectives - Use AI for automated code review and quality enhancement. - Identify and fix syntax, logical, performance, and security issues in Python code. - Improve readability and maintainability through structured refactoring and comments.			Week5 - Monday

- Apply prompt engineering for targeted improvements.
- Evaluate AI-generated suggestions against PEP 8 standards and software engineering best practices

Task Description #1 – Syntax and Logic Errors

Task: Use AI to identify and fix syntax and logic errors in a faulty Python script.

Sample Input Code:

```
# Calculate average score of a student
```

```
def calc_average(marks):
```

```
    total = 0
```

```
    for m in marks:
```

```
        total += m
```

```
    average = total / len(marks)
```

```
    return avrage # Typo here
```

```
marks = [85, 90, 78, 92]
```

```
print("Average Score is ", calc_average(marks))
```

Expected Output:

- Corrected and runnable Python code with explanations of the fixes.

Task Description #2 – PEP 8 Compliance

Task: Use AI to refactor Python code to follow PEP 8 style guidelines.

Sample Input Code:

```
def area_of_rect(L,B):return L*B
```

```
print(area_of_rect(10,20))
```

Expected Output:

- Well-formatted PEP 8-compliant Python code.

Task Description #3 – Readability Enhancement

Task: Use AI to make code more readable without changing its logic.

Sample Input Code:

```
def c(x,y):
```

```
    return x*y/100
```

```
a=200
```

```
b=15
```

```
print(c(a,b))
```

Expected Output:

- Python code with descriptive variable names, inline comments,

and clear formatting.

Task Description #4 – Refactoring for Maintainability

Task: Use AI to break repetitive or long code into reusable functions.

Sample Input Code:

```
students = ["Alice", "Bob", "Charlie"]
print("Welcome", students[0])
print("Welcome", students[1])
print("Welcome", students[2])
```

Expected Output:

- Modular code with reusable functions.

Task Description #5 – Performance Optimization

Task: Use AI to make the code run faster.

Sample Input Code:

```
# Find squares of numbers
nums = [i for i in range(1,1000000)]
squares = []
for n in nums:
    squares.append(n**2)
print(len(squares))
```

Expected Output:

- Optimized code using list comprehensions or vectorized operations.

Task Description #6 – Complexity Reduction

Task: Use AI to simplify overly complex logic.

Sample Input Code:

```
def grade(score):
    if score >= 90:
        return "A"
    else:
        if score >= 80:
            return "B"
```

	<pre> else: if score >= 70: return "C" else: if score >= 60: return "D" else: return "F" </pre> Expected Output: Cleaner logic using elif or dictionary mapping.	
--	---	--

TASK 1:

PROMPT :Use AI to identify and fix syntax and logic errors in a faulty Python script.

Sample Input Code:

Calculate average score of a student

```

def calc_average(marks):
    total = 0
    for m in marks:
        total += m
    average = total / len(marks)
    return avrage # Typo here

```

marks = [85, 90, 78, 92]

print("Average Score is ", calc_average(marks))

CODE :

```

task6.py task1.py X task5.py
task1.py > ...
1  # Calculate average score of a student
2  def calc_average(marks):
3      total = 0 # Initialize total to 0
4      for m in marks:
5          total += m # Add each mark to total
6          average = total / len(marks) # Calculate average
7          return average # Return the computed average
8
9  marks = [85, 90, 78, 92] # List of marks
10 print("Average Score is", calc_average(marks)) # Print the average score

```

OUTPUT :

```
PS C:\Users\HP\Desktop\ai\lab 10> & C:/User
Average Score is 86.25
PS C:\Users\HP\Desktop\ai\lab 10>
```

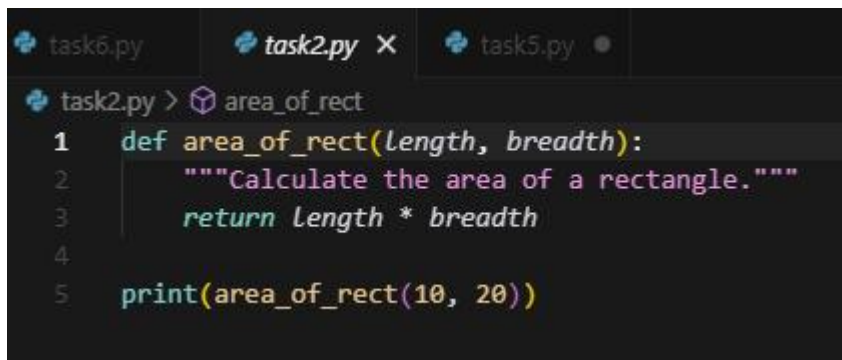
TASK 2:

PROMPT : Use AI to refactor Python code to follow PEP 8 style guidelines.

Sample Input Code:

```
def area_of_rect(L,B):return L*B
print(area_of_rect(10,20))
```

CODE :

A screenshot of a code editor with a dark theme. At the top, there are tabs for 'task6.py', 'task2.py' (which is active and has a close button), and 'task5.py'. Below the tabs, the editor shows the file 'task2.py' with a cursor at the top. The code is as follows:

```
1 def area_of_rect(length, breadth):
2     """Calculate the area of a rectangle."""
3     return length * breadth
4
5 print(area_of_rect(10, 20))
```

OUTPUT :

```
-----
PS C:\Users\HP\Desktop\ai\lab 10> & C:/U
200
PS C:\Users\HP\Desktop\ai\lab 10>
```

TASK 3:

PROMPT : Use AI to make code more readable without changing its logic.

Sample Input Code:

```
def c(x,y):
    return x*y/100
a=200
b=15
print(c(a,b))
```

CODE :

```
task6.py task3.py X task5.py
task3.py > calculate_percentage
1 def calculate_percentage(amount, percent):
2     """Calculate the percentage value of a given amount."""
3     return amount * percent / 100 # Multiply amount by percent and divide by 100
4
5 total_amount = 200 # The base amount to calculate percentage from
6 percentage_value = 15 # The percentage rate to apply
7
8 result = calculate_percentage(total_amount, percentage_value) # Get the percentage value
9 print(result) # Output the result to the console
```

OUTPUT :

```
PS C:\Users\HP\Desktop\ai\lab 10> & C:/User
30.0
PS C:\Users\HP\Desktop\ai\lab 10>
```

TASK 4:

PROMPT : Use AI to break repetitive or long code into reusable functions.

Sample Input Code:

```
students = ["Alice", "Bob", "Charlie"]
print("Welcome", students[0])
print("Welcome", students[1])
print("Welcome", students[2])
```

CODE :

```
task6.py task4.py X task5.py
task4.py > welcome_student
1 def welcome_student(name):
2     """Print a welcome message for a student."""
3     print("Welcome", name)
4
5 students = ["Alice", "Bob", "Charlie"]
6
7 for student in students:
8     welcome_student(student)
```

OUTPUT :

```
at Microsoft.PowerShell.PSConsoleReadLine
-----
PS C:\Users\HP\Desktop\ai\lab 10> & C:/Users
Welcome Alice
Welcome Bob
Welcome Charlie
PS C:\Users\HP\Desktop\ai\lab 10>
```

TASK 5:

PROMPT : Use AI to make the code run faster.

Sample Input Code:

Find squares of numbers

```
nums = [i for i in range(1,1000000)]
```

```
squares = []
```

```
for n in nums:
```

```
    squares.append(n**2)
```

```
print(len(squares))
```

CODE :

```
task6.py task4.py task5.py X
task5.py > ...
1  Ctrl+L to chat, Ctrl+K to generate
2
3  import time
4  start_time = time.time() #start time measurement
5  nums = [i for i in range(1, 1000000)] #list of numbers from 1 to 1000000
6  squares = [n ** 2 for n in nums] #using list comprehension to square each number
7  print(len(squares))
8  end_time = time.time() (variable) end_time: float
9  print(f"Time taken: {end_time - start_time} seconds")
10
```

OUTPUT :

```
-----
PS C:\Users\HP\Desktop\ai\lab 10> & C:/Users/HP/App
999999
Time taken: 0.13889193534851074 seconds
PS C:\Users\HP\Desktop\ai\lab 10>
```

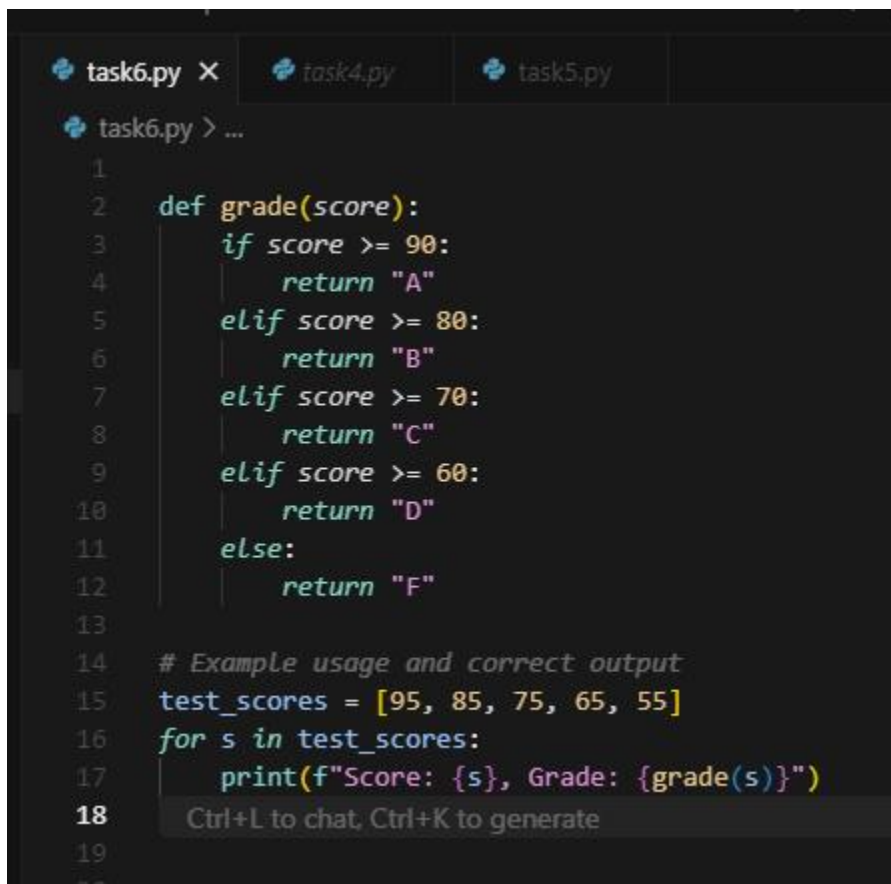
TASK 6 :

PROMPT : Use AI to simplify overly complex logic.

Sample Input Code:

```
def grade(score):  
    if score >= 90:  
        return "A"  
    else:  
        if score >= 80:  
            return "B"  
        else:  
            if score >= 70:  
                return "C"  
            else:  
                if score >= 60:  
                    return "D"  
                else:  
                    return "F"
```

CODE :

A screenshot of a code editor with a dark theme. The editor has three tabs at the top: 'task6.py' (active), 'task4.py', and 'task5.py'. The 'task6.py' tab shows a Python function 'def grade(score):' with a series of 'if' and 'elif' statements to assign grades 'A' through 'F' based on scores. Below the function, there is a comment '# Example usage and correct output' followed by a list 'test_scores = [95, 85, 75, 65, 55]' and a 'for' loop that prints the score and grade for each. A tooltip at the bottom of the editor says 'Ctrl+L to chat, Ctrl+K to generate'.

```
task6.py X task4.py task5.py  
task6.py > ...  
1  
2 def grade(score):  
3     if score >= 90:  
4         return "A"  
5     elif score >= 80:  
6         return "B"  
7     elif score >= 70:  
8         return "C"  
9     elif score >= 60:  
10        return "D"  
11    else:  
12        return "F"  
13  
14 # Example usage and correct output  
15 test_scores = [95, 85, 75, 65, 55]  
16 for s in test_scores:  
17     print(f"Score: {s}, Grade: {grade(s)}")  
18  
19  
20  
Ctrl+L to chat, Ctrl+K to generate
```


OUTPUT :

```
-----  
PS C:\Users\HP\Desktop\ai\lab 10> & C:/User  
Score: 95, Grade: A  
Score: 85, Grade: B  
Score: 75, Grade: C  
Score: 65, Grade: D  
Score: 55, Grade: F  
PS C:\Users\HP\Desktop\ai\lab 10>
```